Implementación de Árboles y Grafos en un Sistema de Gestión de Biblioteca

Actividad 6 - Proyecto final

Yeraldyn Aros Montealegre

Programa de Ingeniería de Software

Universidad Iberoamericana Virtual

Estructuras de Datos

Unidad de Aprendizaje III

Carmen Rubio

11 de octubre de 2026

Implementación de Árboles y Grafos en un Sistema de Gestión de Biblioteca

Las estructuras de datos determinan qué tan rápido y qué tan ordenado se puede acceder a la información dentro de un programa. Las estructuras lineales, como los arreglos, las listas enlazadas, las pilas y las colas, organizan los datos en secuencia; las no lineales, como los árboles y los grafos, permiten representar jerarquías y relaciones más complejas (Zohonero y Joyanes, 2008).

Este documento corresponde a la actividad final del proyecto de la biblioteca. En ella se usan árboles binarios de búsqueda para mejorar la eficiencia en la búsqueda de libros y usuarios, y un grafo para modelar las interacciones entre ellos, es decir, los préstamos. Se presentan el análisis de necesidades, la selección del tipo de grafo, su diseño, su implementación en Java, las pruebas, las optimizaciones y la relación entre las estructuras lineales y no lineales empleadas. El prototipo funcional se entrega en el archivo
Actividad6.java.

Análisis de Necesidades

El sistema debe permitir registrar usuarios y libros, prestar y devolver libros, consultar el estado de cada libro y conservar un historial de movimientos. En estas operaciones aparecen preguntas recurrentes: ¿está prestado este libro?, ¿quién lo tiene?, ¿qué libros tiene un usuario? Todas son preguntas sobre la *relación* entre dos tipos de entidades.

Un grafo es adecuado porque está formado por nodos y conexiones (Fritelli et al., 2020, pp. 355-359), y el préstamo es precisamente una conexión entre un usuario y un libro. Con una lista de adyacencia, saber si un libro está prestado se reduce a revisar si su nodo tiene vecinos,

sin recorrer todos los préstamos del sistema. La Tabla 1 resume las necesidades y la estructura elegida para cada una.

Tabla 1

Necesidades del sistema y estructura de datos elegida

Necesidad	Estructura	Justificación
Buscar libros y usuarios por código	Árbol binario de búsqueda	Búsqueda promedio de orden logarítmico y recorrido inorden ordenado.
Relacionar usuarios con libros prestados	Grafo con listas de adyacencia	Consulta directa de los vecinos de un nodo; permite agregar y quitar préstamos.
Registrar los movimientos del sistema	Pila	El movimiento más reciente se muestra primero (LIFO).
Atender a quien espera un libro prestado	Cola	El primer usuario en pedir el libro es el primero en recibirlo (FIFO).

Selección del Tipo de Grafo

Se eligió un **grafo no dirigido y no ponderado**. Es no dirigido porque la relación de préstamo se consulta en ambos sentidos: desde el usuario se necesitan sus libros y desde el libro se necesita saber quién lo tiene. Por eso la arista se guarda en las dos listas de adyacencia. Es no ponderado porque el préstamo no tiene un costo o distancia asociado; basta con saber si la relación existe o no.

Un grafo dirigido habría exigido mantener dos recorridos distintos (usuario hacia libro y libro hacia usuario). Un grafo ponderado solo se justificaría si se quisiera guardar, por ejemplo, la duración del préstamo como peso de la arista; esa mejora queda como trabajo futuro.

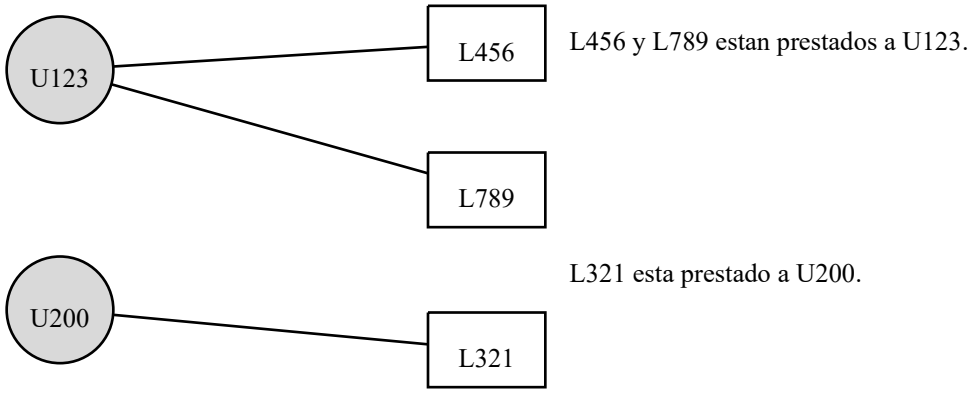
Diseño del Grafo

Los nodos representan usuarios y libros. Para no confundir un usuario con un libro que tenga el mismo código, cada nodo se identifica con un prefijo: *U* para usuarios (por ejemplo, U123) y *L* para libros (por ejemplo, L456). Las aristas representan los préstamos activos. El grafo es bipartito: las aristas solo conectan un usuario con un libro, nunca dos usuarios ni dos libros entre sí.

La representación es una lista de adyacencia implementada con un *HashMap* cuya clave es el nodo y cuyo valor es una *LinkedList* con sus vecinos. Se prefirió a una matriz de adyacencia porque la mayoría de los libros y usuarios no tienen préstamos al mismo tiempo; la matriz reservaría espacio para todas las combinaciones posibles, mientras que la lista solo guarda las aristas existentes.

Figura 1

Ejemplo del grafo de préstamos



Nota. Los círculos son usuarios y los rectángulos son libros. Elaboración propia.

Implementación

El sistema se programó en Java con Apache NetBeans, en un solo archivo, *Actividad6.java*. La clase *Grafo* ofrece las operaciones que pide la actividad: insertar nodos, insertar aristas, buscar y eliminar nodos y aristas. La Tabla 2 las relaciona con su uso dentro de la biblioteca.

Tabla 2

Operaciones del grafo y su uso en el sistema

Operación	Método	Uso en la biblioteca
Insertar nodo	agregarNodo(nodo)	Al registrar un usuario o un libro.
Insertar arista	conectar(a, b)	Al prestar un libro a un usuario.
Buscar arista	estanConectados(a, b)	Al devolver: verifica que ese usuario tenga ese libro.
Consultar vecinos	vecinos(nodo)	Saber si un libro está prestado y qué libros tiene un usuario.
Eliminar arista	desconectar(a, b)	Al devolver un libro.
Eliminar nodo	eliminarNodo(nodo)	Al eliminar un usuario o un libro (solo si no tiene préstamos).

A continuación se muestra la clase del grafo. Al conectar o desconectar, se actualizan las dos listas de adyacencia para mantener la relación en ambos sentidos.

Código 1

Clase Grafo

```
static class Grafo {    private final Map<String, LinkedList<String>> adyacencia
= new HashMap<>();

    void        agregarNodo(String        nodo)        {
adyacencia.putIfAbsent(nodo, new LinkedList<>());     }

    void conectar(String a, String b) {
adyacencia.get(a).add(b);
adyacencia.get(b).add(a);     }
```

```
void desconectar(String a, String b) {
adyacencia.get(a).remove(b);
adyacencia.get(b).remove(a);     }

    boolean estanConectados(String a, String b) {
LinkedList<String>    vecinos    =    adyacencia.get(a);
return vecinos != null && vecinos.contains(b);     }

    LinkedList<String>    vecinos(String    nodo)    {
LinkedList<String>    v    =    adyacencia.get(nodo);
return (v == null) ? new LinkedList<>() : v;     }

    void      eliminarNodo(String      nodo)      {
LinkedList<String> v = adyacencia.get(nodo);       if
(v == null) return;          for (String vecino : v)
{
            adyacencia.get(vecino).remove(nodo);
}
        adyacencia.remove(nodo);
    }
}
```

El préstamo funciona así: si el nodo del libro ya tiene un vecino, el libro está prestado y no se puede prestar de nuevo; si no tiene vecinos, se conecta el nodo del usuario con el nodo del libro, se agrega el movimiento a la pila del historial y se muestra el mensaje con el título del libro y el nombre del usuario. Si el libro estaba prestado, el usuario puede quedar en una cola de espera, y al devolverse el libro se presta automáticamente al primero de la cola.

Los libros y usuarios se guardan además en árboles binarios de búsqueda. Cada nodo del árbol se compara por el código: si el código buscado es menor, se continúa por la izquierda; si es mayor, por la derecha; si es igual, se encontró el elemento. El árbol también permite eliminar con sus tres casos (sin hijos, con un hijo y con dos hijos) y recorrer los datos en orden con el recorrido inorden.

Árbol binario de búsqueda

Para buscar libros y usuarios por código se implementó un árbol binario de búsqueda genérico. Cada nodo guarda un código y el objeto asociado; los códigos menores van al subárbol izquierdo y los mayores al derecho. Así, en cada comparación se descarta aproximadamente la mitad de

los elementos restantes cuando el árbol está equilibrado, en lugar de revisar uno por uno como en una lista (Zohonero y Joyanes, 2008).

Código 2

Inserción y búsqueda en el árbol

```
boolean insertar(int clave, T dato) {       Nodo<T>
nuevo = new Nodo<>(clave, dato);    if (raiz == null)
{ raiz = nuevo; return true; }      Nodo<T> actual =
raiz;     while (true) {
        if (clave == actual.clave) return false;       // codigo repetido
if (clave < actual.clave) {                    // izquierda
            if (actual.izquierda == null) { actual.izquierda = nuevo; return true; }
actual = actual.izquierda;         } else {                                        //
derecha
            if (actual.derecha == null) { actual.derecha = nuevo; return true; }
actual = actual.derecha;         }
    }
}

T   buscar(int   clave)   {
Nodo<T>   actual   =   raiz;
while  (actual  !=  null)  {
if  (clave  ==  actual.clave)
return          actual.dato;
actual     =     (clave     <
actual.clave)              ?
actual.izquierda           :
actual.derecha;            }
return                 null;
// no existe }
```

La inserción rechaza los códigos repetidos, la búsqueda devuelve *null* si el código no existe y el recorrido inorden entrega los datos ordenados por código, que es lo que usa la opción de consulta.

Pruebas y Validación

Se diseñaron casos de prueba para cada operación del menú, incluyendo los casos de error. La Tabla 3 indica el resultado que debe mostrar el programa en cada caso.

Tabla 3

Casos de prueba y resultado esperado

N.º	Caso de prueba	Resultado esperado
1	Prestar sin usuarios ni libros registrados	Mensaje de que no se puede prestar.
2	Ingresar un campo vacío o un número negativo	Se rechaza el dato y se vuelve a pedir.
3	Registrar un usuario con un código ya existente	Mensaje de que el código ya existe.
4	Prestar con un código de usuario o de libro inexistente	Mensaje de que no existe.
5	Prestar un libro disponible	Se crea la arista y se muestra el mensaje del préstamo.
6	Prestar un libro que ya está prestado	Mensaje de que está prestado y oferta de lista de espera.
7	Devolver un libro que el usuario no tiene	Mensaje de que ese usuario no tiene ese libro.
8	Devolver un libro con lista de espera	Se elimina la arista y se presta al siguiente de la cola.
9	Consultar con préstamos activos	Se listan usuarios, libros con su estado, préstamos e historial.
10	Eliminar un libro prestado	No se permite eliminarlo.
11	Eliminar un libro disponible	Se elimina del árbol y del grafo.

Optimización

La Tabla 4 resume el costo de las operaciones principales, donde n es el número de elementos del árbol y g es el número de vecinos de un nodo. Con la lista de adyacencia el espacio utilizado es proporcional a la cantidad de nodos más la cantidad de aristas, por lo que solo se almacenan los préstamos que realmente existen.

Tabla 4

Costo aproximado de las operaciones

Operación	Estructura	Costo
Buscar por código	Árbol binario de búsqueda	Promedio O(log n); peor caso O(n)
Saber si un libro está prestado	Grafo (lista de adyacencia)	O(1) en promedio al localizar el nodo; el libro tiene como máximo un vecino
Conectar o desconectar	Grafo	O(g)
Registrar un movimiento	Pila	O(1)
Atender la cola de espera	Cola	O(1)

Como mejoras, se evitó recorrer todas las listas de préstamos para saber si un libro está disponible, y se impidió eliminar usuarios o libros con préstamos activos para mantener el grafo consistente. Una limitación es que el árbol binario de búsqueda no se balancea: si los códigos se registran en orden creciente, el árbol se degenera en una lista y la búsqueda pasa a ser de orden lineal. Para evitarlo se podría usar un árbol AVL (Zohonero y Joyanes, 2008).

Uso y Relación de Conceptos

El proyecto combina estructuras lineales y no lineales, y cada una resuelve una parte distinta del problema. Las estructuras lineales organizan los datos en secuencia: la lista enlazada guarda los vecinos de cada nodo, la pila guarda el historial con el último movimiento primero (LIFO), la cola atiende la lista de espera en el orden de llegada (FIFO) y el arreglo dinámico almacena los datos ordenados al momento de mostrarlos.

Las estructuras no lineales representan información que no sigue una sola secuencia. El árbol organiza los datos de forma jerárquica, desde un nodo raíz con ramas hacia la izquierda y la derecha, lo que facilita la búsqueda. El grafo, formado por nodos y conexiones, representa

relaciones más complejas como el préstamo entre usuarios y libros. Ambos se complementan: el árbol encuentra rápidamente el usuario o el libro, y el grafo indica cómo se relacionan.

Elegir la estructura adecuada hace que las búsquedas sean más rápidas y que los problemas complejos se representen de forma más sencilla; además, obliga a pensar no solo en guardar la información, sino en cómo recorrerla, buscarla, modificarla y relacionarla.

Prototipo Funcional

El prototipo es la aplicación de consola *Actividad6.java*, que se ejecuta desde Apache NetBeans. Al iniciar muestra un menú con las opciones de la Tabla 5.

Tabla 5

Opciones del menú del prototipo

Opción	Función
1. Registrar usuario	Guarda el usuario en el árbol y crea su nodo en el grafo.
2. Registrar libro	Guarda el libro en el árbol y crea su nodo en el grafo.
3. Prestar libro	Valida los datos, crea la arista usuario-libro o ofrece la lista de espera.
4. Devolver libro	Elimina la arista y presta al siguiente usuario en espera, si lo hay.
5. Consultar	Muestra usuarios, libros con su estado, préstamos activos e historial.
6. Buscar	Busca un libro o usuario por código y muestra su relación en el grafo.
7. Eliminar	Elimina un libro o usuario sin préstamos del árbol y del grafo.
0. Salir	Termina el programa.

Conclusiones

Los árboles binarios de búsqueda permitieron encontrar libros y usuarios por código sin recorrerlos todos, y el grafo permitió representar los préstamos de forma directa: cada préstamo es una arista y cada consulta sobre el estado de un libro o de un usuario se resuelve revisando los vecinos de un nodo. La decisión de usar un grafo no dirigido y no ponderado se ajustó al problema, porque la relación se consulta en ambos sentidos y no tiene un costo asociado.

Combinar el árbol binario de búsqueda, el grafo, la pila y la cola muestra que ninguna estructura resuelve todo por sí sola, sino que cada una se elige según la operación que debe hacerse con los datos. Como trabajo futuro, se propone balancear el árbol con un AVL y ponderar las aristas con la fecha de devolución para controlar los préstamos vencidos.

Referencias

Fritelli, V., Guzmán, A., & Tymoschuk, J. (2020). *Algoritmos y estructuras de datos* (2.ª ed.). Jorge Sarmiento Editor - Universitas.

Oracle. (s. f.). *Java Platform, Standard Edition API specification*. https://docs.oracle.com/en/java/javase/21/docs/api/index.html

Sánchez Juárez, J. (2013). *Libro digital Estructuras de datos*. Libre Edición.

Zohonero Martínez, I., & Joyanes Aguilar, L. (2008). *Estructuras de datos en Java*. McGraw-Hill.